# Apache Spark Interview Guide : Core Concepts

- ☑ Jobs, Stage and Task
- ☑ RDDs and Dataframes
- ☑ PySpark and Spark JVM
- ☑ Spark Job Submission Workflow

De sarthkumar

# Job, Stage and Tasks

The concepts of job, stage, and tasks are fundamental to understanding how Spark manages distributed computing. These elements are part of the execution model that Spark uses to distribute and manage computation across multiple nodes efficiently. Let's break down what each of these terms means and how they relate to each other in the context of executing a PySpark program.

## Job

A job in PySpark is the entire process needed to complete a computation initiated by an action. When you execute an action (like `collect()`, `count()`, `save()`, etc.) on a RDD (Resilient Distributed Dataset) or a DataFrame, Spark submits a job. The job represents the entire piece of computation or query from start to finish. For example, if you have a PySpark DataFrame and you want to count the number of rows that satisfy a particular condition, you might use the `count()` action after a `filter()` transformation. The execution of this `count()` action triggers a job.

Logical Unit of Work: A job in Spark represents a complete computation triggered by an action on a RDD or DataFrame. Jobs are composed of one or more stages, each containing multiple tasks. They provide a high-level abstraction for users to define and manage computations on distributed datasets.

Dependency Management: Jobs manage the dependencies between transformations and actions in a Spark application. Spark constructs a directed acyclic graph (DAG) of operations, where each node represents a transformation, and each



Deepa Vasanthkumar

edge represents a dependency. Jobs orchestrate the execution of these operations in the correct order, ensuring consistency and correctness of the computation.

Optimization and Monitoring: Jobs enable Spark to optimize the execution plan by applying various optimizations, such as task scheduling, data locality, and memory management. Users can monitor job execution through the Spark UI and control job submission and execution parameters programmatically or through configuration settings.

## Stage

A stage is a set of tasks in a job that can be executed in parallel but must be completed before the next set of tasks (the next stage) can begin. Stages in a Spark job are created based on the transformations applied to the RDD or DataFrame. Specifically, stages are divided by transformations that cause shuffling of data across partitions, such as `groupBy()` or `reduceByKey()`.

Shuffle boundaries (where data must be redistributed across different nodes) are a common reason for new stages. Each stage consists of tasks that execute the same code on different partitions of the data, and all tasks in a stage must be completed before any tasks in subsequent stages start.



Deepa Vasanthkumar

Execution Pipelining: Stages are logical units of work in Spark that represent a series of transformations that can be executed together without shuffling data between tasks. Spark breaks down a computation into stages based on data dependencies and transformations. By organizing tasks into stages, Spark can optimize the execution plan by pipelining transformations and minimizing data shuffling.

Shuffle Boundary: Stages are often delineated by operations that require data shuffling, such as `groupByKey()` or `join()`. When a shuffle occurs, Spark creates a new stage, allowing it to manage data redistribution efficiently. Stages help in managing the complexity of distributed computations and enable Spark to optimize performance by minimizing shuffle operations.

## Task

A task is the smallest unit of work that gets sent to an executor. Essentially, a task applies the job's logic to a single partition of the data. Tasks within a stage perform the same operations on different pieces of the data and can be executed in parallel across different executors and nodes in the Spark cluster. Tasks are what actually execute the code on the cluster.

Parallel Execution: Tasks are the basic units of computation in Spark. They allow for parallel execution of operations on distributed data partitions across multiple executor nodes in a cluster. By breaking down computations into smaller tasks, Spark can utilize the processing power of the entire cluster efficiently.



Deepa Vasanthkumar

Data Partitioning: Tasks operate on individual partitions of the distributed datasets. This partitioning enables Spark to process large datasets that may not fit into the memory of a single machine. Each task processes a subset of the data independently, facilitating parallelism and scalability.

Fault Tolerance: Spark achieves fault tolerance at the task level. If a task fails during execution (due to hardware failures, network issues, etc.), Spark can rerun that specific task on another node using the data lineage information. By isolating failures to individual tasks, Spark minimizes the impact of failures on the overall computation.

Demonstrating with an example:

Here's a simple example to illustrate how jobs, stages, and tasks might be structured in a PySpark program:

Deepa Vasanthkumar

```
file_path = "file in s3 or local "

data = spark.read.text(file_path)

words = data.rdd.flatMap(lambda line: line.value.split())

words_starting_with_a = words.filter(lambda word: word.startswith('a'))

count = words_starting_with_a.count()

print("Number of words starting with 'a':", count)
```

Deepa Vasanthkumar

1. RDD Creation: Suppose you read in a large dataset into an RDD.

2. Transformation: You apply a `map()` transformation to modify the data.

3. Another Transformation: You then apply a `filter()` transformation to remove some rows.

4. Action: Finally, you call `count()` to get the number of elements in the transformed RDD.

In this example:

- The call to `count()` triggers a job.

- This job may consist of one stage if there's no shuffle needed (if the transformations only involve operations like `map()` and `filter()` that do not necessitate a shuffle).

- The stage will have as many tasks as there are partitions in the RDD. Each task processes one partition of the data independently on different executors.

If there was a shuffle transformation involved (like `reduceByKey()`), then there would be at least two stages:



Deepa Vasanthkumar

- One stage to process the transformations before the shuffle.

- Another stage to process the shuffle and any transformations after the shuffle.

Understanding these components helps in optimizing and debugging PySpark applications, as you can identify where bottlenecks might occur (like excessive shuffling or uneven task distribution) and tune your Spark application accordingly.

In PySpark, jobs that involve a shuffle operation are more complex in terms of how they are divided into stages and tasks. This complexity arises because shuffle operations require redistributing data across different nodes in the cluster to group it differently according to the keys involved in the shuffle. Let's delve into how stages and tasks are structured when a shuffle operation is part of a PySpark job.

Deepa Vasanthkumar

## Shuffle in Spark

A shuffle occurs when data needs to be redistributed across different executor nodes in a Spark cluster. This redistribution is often necessary when performing "wide" transformations such as `groupByKey()`, `reduceByKey()`, `join()`, and others that involve grouping or aggregating data across partitions. The shuffle operation involves writing data to disk and transferring it over the network, making it one of the most resource-intensive operations in Spark.

### Stages in Jobs with Shuffle

In jobs that involve a shuffle, stages are generally divided around the shuffle points:

1. Pre-Shuffle Stage: This is the stage before the shuffle occurs. In this stage, tasks perform transformations on the data within their respective partitions. These transformations are "narrow," meaning they do not require data from other partitions. Examples include `map()`, `filter()`, and `flatMap()`.

2. Shuffle Write: At the end of the pre-shuffle stage, tasks write their results to disk. This output is organized such that data destined for the same partition in the next stage is grouped together.

Deepa Vasanthkumar

3. Shuffle Read: The next stage begins with a shuffle read. Executors fetch the relevant parts of data from across all nodes. This is where the heavy lifting of shuffle happens: data needs to be pulled from potentially all other nodes in the cluster, depending on the keys involved.

4. Post-Shuffle Stage: After the shuffle read, the next set of tasks can begin. These tasks operate on the newly formed partitions, which may now contain data from multiple original partitions. The operations in this stage are also "wide," meaning they generally involve data that needs to be aggregated or joined across the original partitions.

### Tasks in Jobs with Shuffle

Tasks are the units of execution that carry out the computation:

1. Pre-Shuffle Tasks: Each task in the pre-shuffle stages processes a partition of the data and performs narrow transformations. The results are then written to local disks in a format ready for shuffling.

2. Shuffle Write Tasks: As part of the end of a pre-shuffle task, the data is organized and written out to disk to prepare for shuffling.

3. Shuffle Read Tasks: During the shuffle read phase, each task is responsible for fetching the appropriate data segments from the shuffle write outputs of tasks in the previous stage.



Deepa Vasanthkumar

4. Post-Shuffle Tasks: These tasks process the gathered data. For example, if the transformation is `reduceByKey()`, each task now has data from all partitions related to specific keys and can perform the reduction operation.

# RDD & Dataframes

Apache Spark offers two fundamental data abstractions: Resilient Distributed Datasets (RDDs) and DataFrames. Let's delve into each of these core concepts:

## Resilient Distributed Datasets (RDDs):

RDDs are the foundational data structure in Apache Spark, representing an immutable distributed collection of objects that can be operated on in parallel.

They are fault tolerant, meaning if a partition of an RDD is lost due to node failure, Spark can recreate it through lineage (the series of transformations applied to the base data).

RDDs can be created from various data sources such as HDFS (Hadoop Distributed File System), HBase, or by parallelizing existing collections in memory.

RDDs support two types of operations: transformations and actions. Transformations create a new RDD from an existing one (e.g., `map`, `filter`, `reduceByKey`), while actions compute a result based on an RDD and either return it to the driver program or save it to an external storage system (e.g., `count`, `collect`, `saveAsTextFile`).

While RDDs offer low level control and flexibility, they lack optimization opportunities provided by higher level abstractions like DataFrames.

Deepa Vasanthkumar

## DataFrames:

DataFrames are distributed collections of structured data organized into named columns, inspired by data frames in R and Python (Pandas).

They provide a higher level abstraction compared to RDDs, offering a domain specific language (DSL) for manipulating structured data.

DataFrames are built on top of RDDs but provide additional optimizations through Spark's Catalyst Optimizer and Tungsten Execution Engine.

DataFrames support a wide range of operations including filtering, aggregating, joining, and windowing, and they seamlessly integrate with Spark SQL for querying structured data using SQL syntax.

DataFrames can be created from various sources such as Hive tables, Apache Parquet, JSON, CSV, or from existing RDDs.

Unlike RDDs, DataFrames maintain schema information, enabling Spark to perform optimizations such as predicate pushdown, column pruning, and bytecode generation for better performance.

DataFrames are particularly wellsuited for structured data processing tasks, including ETL (Extract, Transform, Load) pipelines, data exploration, and interactive analytics.

**Abstraction Level:**

RDDs: RDDs provide a low level abstraction for distributed data processing. They offer fine grained control over data manipulation and are suitable for complex data processing tasks where precise control over transformations and actions is required.

DataFrames: DataFrames offer a higher level abstraction compared to RDDs. They provide a domain specific language (DSL) for manipulating structured data, making it easier to express transformations and queries. DataFrames are more suitable for data processing tasks involving structured data.

**Schema Handling:**

RDDs: RDDs do not have an inherent schema. Each element in an RDD can be of any Java, Scala, or Python object type. Developers have to handle schema validation and parsing manually.



Deepa Vasanthkumar

DataFrames: DataFrames maintain a schema, which defines the structure of the data including column names and data types. Schema enforcement is automatic, making it easier to work with structured data. Spark's Catalyst Optimizer can leverage schema information for query optimization.

**Performance:**

RDDs: RDDs offer lower performance compared to DataFrames due to the lack of optimization opportunities. While RDDs can be optimized through manual tuning and caching, they generally do not benefit from Spark's Catalyst Optimizer and Tungsten Execution Engine.

DataFrames: DataFrames are optimized for performance. They leverage Spark's Catalyst Optimizer and Tungsten Execution Engine, which perform query optimization, bytecode generation, and memory management optimizations. As a result, DataFrames typically offer better performance than RDDs for structured data processing tasks.

**Ease of Use:**

RDDs: Working with RDDs requires a deeper understanding of Spark's programming model and API. Developers need to handle serialization, fault tolerance, and other low level details manually.

DataFrames: DataFrames provide a more user friendly API, resembling SQL or Pandas DataFrames. They offer a higher level of abstraction, making it easier for developers to express data transformations and queries. DataFrames also integrate seamlessly with Spark SQL, allowing users to leverage SQL syntax for data manipulation.

**Integration with Ecosystem:**

RDDs: RDDs are the foundational data abstraction in Spark and can be used with all Spark components and libraries. They offer flexibility but may require more effort for integration with higher level components.

DataFrames: DataFrames are widely supported across the Spark ecosystem. They can be seamlessly integrated with Spark SQL, MLlib (Machine Learning library), and other Spark components. Many libraries and tools in the Spark ecosystem are designed to work with DataFrames, making them a preferred choice for interoperability.



Deepa Vasanthkumar

# PySpark and Spark JVM

When you're working with PySpark, which is the Python API for Apache Spark, your Python code interacts with the Spark JVM (Java Virtual Machine) underneath. When you create RDDs in PySpark, they are essentially Python wrappers around Java RDDs.

**Python to Java Communication:**

Python code interacts with the PySpark API, which internally communicates with the Java Virtual Machine (JVM) where Spark runs.

When you create RDDs or perform transformations/actions on them in PySpark, these operations are translated into equivalent Java operations in the Spark JVM.

**Serialization:**

Python objects, such as lists or dictionaries, need to be serialized before they can be sent to the JVM.

PySpark uses a mechanism called Py4J for communication between Python and Java. Py4J handles the serialization and deserialization of Python objects to/from Java objects.

**Execution in Java:**

Once the Python operations are translated into Java operations and the data is serialized, the actual computation happens in the Spark JVM.

Java RDDs are created and manipulated based on the operations defined in the Python code.

**Data Movement:**

When data needs to be shuffled or moved between partitions, it is done in the JVM.

Data movement operations such as shuffling are executed efficiently within the JVM, leveraging Spark's distributed computing capabilities.

5. Results Retrieval:



Deepa Vasanthkumar

When an action is triggered in PySpark (e.g., `collect()` or `count()`), the results are computed in the JVM.

The results are then serialized back to Python objects using Py4J and returned to the Python environment for further processing or output.

RDDs created in Python are essentially wrappers around Java RDDs, and the actual computation happens in the Java Virtual Machine. PySpark handles the translation of Python operations into equivalent Java operations, serialization of data, and communication between the Python and Java environments.

# Spark Job Submission Workflow

When you submit a job in Apache Spark, the following sequence of events takes place:

**Driver Program:** The job starts with the driver program, which contains your Spark application code. The driver is responsible for the following steps:

SparkContext Creation: Initializes the SparkContext, which is the main entry point for Spark functionality. This sets up internal services and establishes a connection to the cluster manager.

RDD/Dataset/DataFrame Creation: Constructs the logical plan of the operations you want to perform on your data.

**Job Submission:**

Job Creation: The driver program translates the logical plan into a physical plan, consisting of stages and tasks.



Deepa Vasanthkumar

Task Scheduling: The physical plan is divided into tasks that can be executed across the cluster nodes. These tasks are sent to the cluster manager (YARN, Mesos, or Kubernetes) to be scheduled on executor nodes.

**Cluster Manager:** The cluster manager allocates resources (CPU, memory) across the worker nodes and launches executors.

**Executor Nodes:**

Task Execution: Executors execute the tasks as directed by the driver program.

Data Storage: Executors read and write data to and from disk or memory as needed.

**Result Collection:** Once tasks are completed, the results are sent back to the driver. The driver may perform additional transformations or actions based on the returned results.

**Job Completion:** The driver program completes execution after collecting and processing the results.

## Databricks Notebook: Job Execution Workflow

Databricks notebooks provide an interactive environment built on top of Apache Spark, with additional features and integrations:

Databricks notebooks allow users to write and execute code in an interactive, web-based environment. Users can write code in multiple languages, including Python, Scala, SQL, and R.

Cluster Association: The notebook is associated with an active cluster. If no cluster is active, Databricks automatically starts a new one based on the notebook's configuration.

Driver and Executors: Similar to standalone Spark, the driver program is responsible for orchestrating the job, and executors run the tasks. The driver and executors are part of the Databricks cluster.



Deepa Vasanthkumar

Immediate Feedback: As each cell is executed, the results are immediately displayed within the notebook. This provides an interactive development experience.

Job Management: Dtabricks abstracts away the complexity of job scheduling and resource management, making it easier for users to focus on writing and optimizing their Spark jobs.
Auto-Scaling: Clusters can be configured to auto-scale based on the workload, providing efficient resource utilization.

Real-time Collaboration: Multiple users can collaborate on the same notebook in real-time.
Version Control: Notebooks are versioned automatically, allowing users to track changes and revert to previous versions if necessary.

Data Sources: Databricks integrates with various data sources, including cloud storage (e.g., AWS S3, Azure Blob Storage), databases, and data lakes.
ML and BI Tools: Databricks notebooks integrate seamlessly with machine learning frameworks (e.g., MLflow) and business intelligence tools (e.g., Power BI, Tableau).

## Key Differences between spark-submit and notebook

- Execution Context: Spark jobs are typically submitted through a script or a command-line interface, while Databricks jobs are run interactively within a notebook environment.
- Resource Management: Databricks abstracts much of the resource management and cluster configuration, offering features like auto-scaling and automated cluster management.
- User Experience: Databricks provides a more user-friendly and collaborative interface, with real-time feedback and integration capabilities that extend beyond traditional Spark applications.
- Convenience Features: Databricks includes additional features such as version control, interactive dashboards, and seamless integration with other data and machine learning tools.



Deepa Vasanthkumar